

Task-Priority kinematic control (2B)

Lab 6

Course: Hands-on Intervention

Date: March 30, 2025

Pravin Oli

u1999699, pravin.oli.08@gmail.com

code patner: Gebreslassie Gebrecherkos Gebrezgabhier

u1999542, chereq2016@gmail.com

Abstract

This lab focused on extending the Recursive Task-Priority (TP) kinematic control framework to a mobile manipulator system, combining a differential-drive base with a 3-link planar robotic arm. A new `MobileManipulator` class was developed to integrate the base and arm into a unified control structure. The TP controller was adapted to handle multiple task hierarchies, including position control, joint constraints, and configuration tracking.

In Exercise 1, the system was tested on random end-effector targets while enforcing joint velocity limits using set-based inequality tasks. Exercise 2 introduced the Weighted Damped Least-Squares (WDLS) method to prioritize motion contributions from specific joints or the mobile base. Exercise 3 compared three mobile base integration strategies—move forward then rotate, rotate then move forward, and simultaneous arc-based motion—to evaluate their effects on end-effector accuracy and trajectory smoothness.

The results demonstrated successful task execution and control stability across all exercises, highlighting the effectiveness of TP control for mobile manipulators in dynamic and constrained environments.

Contents

1	Introduction	3
2	Objectives	3
3	Mobile Base Kinematics	3
3.1	Pose Representation	3
3.2	Kinematic Update Models	4
3.2.1	Forward Then Rotate	5
3.2.2	Rotate Then Forward	5
3.2.3	Simultaneous Rotation and Translation (Arc Motion)	5
3.3	Base-to-World Transformation	5
3.4	Manipulator Mounting Offset	5
3.5	Total Homogeneous Transformation Matrix	6
3.5.1	Transformation Chain for Extended Base Joints	7

4	Implementation	7
4.1	lab2_robotics.py	7
4.2	lab5_robotics.py	7
4.3	lab6_robotics.py	7
4.4	Exercise1.py	7
4.5	Exercise2.py	8
4.6	Exercise3.py	8
4.7	plot.py	8
5	Results and Discussion	8
5.1	Exercise 1: Joint Limit Task and Position Control	8
5.2	Exercise 2: Configuration Control with Weighted DLS	10
5.3	Exercise 3: Comparison of Base Integration Strategies	11
6	Conclusion	12
	Appendix	13
A	lab6_robotics.py	13
B	Exercise1.py	17
C	Exercise2.py	20
D	Exercise3.py	25
E	plot.py	29

1 Introduction

In this lab, we extended the Recursive Task-Priority (TP) kinematic control algorithm to handle mobile manipulators to combine a mobile base with a robotic arm ie. differential-drive robot with a 3-link planar arm mounted on top. A new `MobileManipulator` class that supports base movement and arm control was made compatible with the existing TP control code from previous labs. The robot's motion was guided by a hierarchy of tasks, such as positioning the end-effector or keeping joint movements within limits.

The lab had three main parts:

Exercise 1 focused on implementing the full mobile manipulator model, defined the robot's kinematics and use the TP controller to move the end-effector to randomly chosen targets, while respecting joint constraints.

Exercise 2 introduced the **Weighted Damped Least-Squares (WDLS)** method. By assigning different weights to the robot's degrees of freedom, we could influence whether the mobile base or the manipulator was more active during motion.

Exercise 3 compared different strategies for updating the mobile base's pose—moving forward then rotating, rotating then moving forward, or moving along an arc to analyze how each approach affected the robot's path and control accuracy.

2 Objectives

The primary objectives of this lab session are as follows:

- **To develop a kinematic model of a mobile manipulator** that integrates a differential-drive mobile base with a 3-link planar manipulator, and implement it within the existing Task-Priority control framework.
- **To investigate the impact of weighting strategies** in the control algorithm using the Weighted Damped Least-Squares (WDLS) method, allowing the prioritization of specific degrees of freedom within the system.
- **To evaluate different mobile base integration methods** for updating the robot's pose, and analyze how these strategies affect end-effector control accuracy and trajectory smoothness.

3 Mobile Base Kinematics

The mobile base in this lab is modeled as a **differential-drive robot**, which is a common type of non-holonomic mobile platform. It operates in a 2D plane and can perform forward/backward translation and in-place rotation, but cannot move laterally. The robot's pose is described by its position and orientation in the plane.

3.1 Pose Representation

The pose of the base is defined by the vector:

$$\boldsymbol{\eta} = \begin{bmatrix} x \\ y \\ \psi \end{bmatrix}$$

where:

- x : position along the global X-axis,
- y : position along the global Y-axis,
- ψ : orientation (yaw) of the robot in the plane.

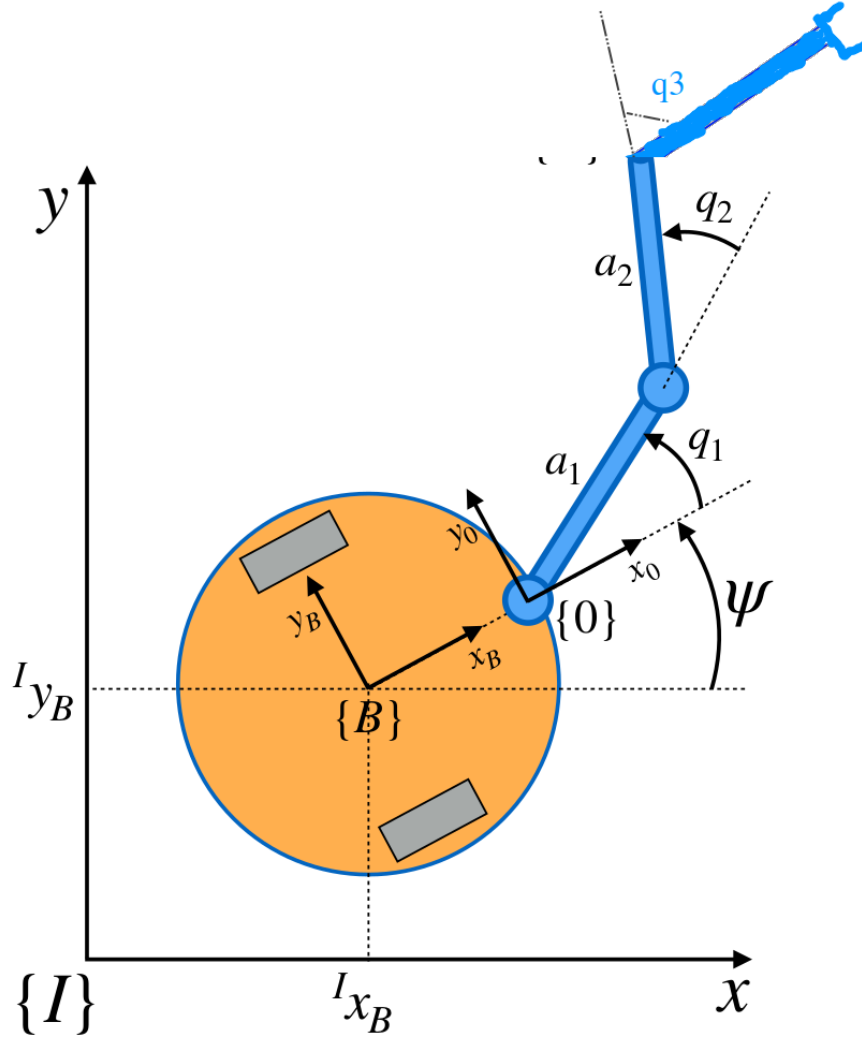


Figure 1: Differential Drive Mobile Base

3.2 Kinematic Update Models

Three different update strategies were implemented to compute the evolution of the base pose over time, using the quasi-velocity inputs:

$$\dot{\eta} = \begin{bmatrix} \omega \\ v \end{bmatrix}$$

where ω is the angular velocity (yaw rate), and v is the linear forward velocity.

The following update equations were implemented:

3.2.1 Forward Then Rotate

$$\begin{aligned}x_{t+1} &= x_t + v \cdot \cos(\psi_t) \cdot dt \\y_{t+1} &= y_t + v \cdot \sin(\psi_t) \cdot dt \\\psi_{t+1} &= \psi_t + \omega \cdot dt\end{aligned}$$

3.2.2 Rotate Then Forward

$$\begin{aligned}\psi_{t+1} &= \psi_t + \omega \cdot dt \\x_{t+1} &= x_t + v \cdot \cos(\psi_{t+1}) \cdot dt \\y_{t+1} &= y_t + v \cdot \sin(\psi_{t+1}) \cdot dt\end{aligned}$$

3.2.3 Simultaneous Rotation and Translation (Arc Motion)

This model captures the real-world behavior of a robot moving along a circular arc when both angular and linear velocities are applied.

$$\begin{aligned}R &= \frac{v}{\omega} \\x_{t+1} &= x_t - R \cdot \sin(\psi_t) + R \cdot \sin(\psi_t + \omega \cdot dt) \\y_{t+1} &= y_t + R \cdot \cos(\psi_t) - R \cdot \cos(\psi_t + \omega \cdot dt) \\\psi_{t+1} &= \psi_t + \omega \cdot dt\end{aligned}$$

3.3 Base-to-World Transformation

The transformation matrix from the mobile base frame to the world frame is given by:

$$T_{base} = \begin{bmatrix} \cos \psi & -\sin \psi & 0 & x \\ \sin \psi & \cos \psi & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This matrix is used as the base transformation T_0 for the forward kinematics of the manipulator, effectively anchoring the robot arm onto a moving platform.

3.4 Manipulator Mounting Offset

The arm is not mounted at the geometric center of the mobile base. A fixed offset r is applied from the robot's center to the manipulator base along the Y-axis of the base frame:

$$T_{offset} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & r \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This offset is appended in the kinematic chain before computing the manipulator's DH transformations.

3.5 Total Homogeneous Transformation Matrix

The full transformation from the world frame to the end-effector frame combines three main components:

1. The pose of the mobile base in the world frame: T_{base} ,
2. The fixed offset from the base center to the manipulator's base: T_{offset} ,
3. The chain of DH transformations representing the 3-link manipulator: T_i^{i-1} .

The total transformation matrix is given by:

$$T_{0E} = T_{\text{base}} \cdot T_{\text{offset}} \cdot \left(\prod_{i=1}^3 T_i^{i-1} \right)$$

Where:

- $T_{\text{base}} \in SE(3)$ is the base-to-world transformation, defined as:

$$T_{\text{base}} = \begin{bmatrix} \cos \psi & -\sin \psi & 0 & x \\ \sin \psi & \cos \psi & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- T_{offset} is the transformation from the robot center to the manipulator base:

$$T_{\text{offset}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & r \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Each T_i^{i-1} is a standard Denavit–Hartenberg transformation matrix:

$$T_i^{i-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i & 0 & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i & 0 & a_i \sin \theta_i \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (\text{since } \alpha_i = 0, d_i = 0)$$

Thus, the final end-effector pose in the world frame is computed as:

$$T_{0E} = T_{\text{base}} \cdot T_{\text{offset}} \cdot T_1^0 \cdot T_2^1 \cdot T_3^2$$

This transformation is also used to compute the Jacobian matrix for end-effector velocity control within the Task-Priority framework.

The combined transformation is used to calculate both the end-effector pose and the Jacobian required for task-space control.

3.5.1 Transformation Chain for Extended Base Joints

The complete kinematic chain starts with the base motion represented as a series of virtual joints (rotations and translations) before reaching the actual manipulator joints:

$$\begin{aligned}\dot{\psi} &\rightarrow R_z(\psi)R_x(-90^\circ), & \dot{\theta} &\rightarrow R_z(\theta + 90^\circ)R_x(90^\circ), & \dot{\phi} &\rightarrow R_z(\phi) \\ v_x &\rightarrow T_z(0)R_x(-90^\circ), & v_y &\rightarrow T_z(0)R_z(-90^\circ)R_x(90^\circ), & v_z &\rightarrow T_z(0) \\ \dot{q}_i &\rightarrow DH(d_i, q_i, a_i, \alpha_i)\end{aligned}$$

This sequence ensures that the Jacobian includes all contributions from the mobile base in a structured and consistent way, aligned with the manipulator's kinematic chain.

4 Implementation

This lab is implemented using the following Python scripts: `lab2_robotics.py` (from the previous lab), `lab5_robotics.py` (from the previous lab), `lab6_robotics.py`, `Exercise1.py`, `Exercise2.py`, `Exercise3.py`, and `plot.py`. The detailed full code is in **Appendix section**

4.1 lab2_robotics.py

This is the same as in the previous lab.

4.2 lab5_robotics.py

This is the same as in the previous lab.

4.3 lab6_robotics.py

- Defines a new `MobileManipulator` class extending the `Manipulator`.
- Adds a mobile base with position and orientation (`x`, `y`, `yaw`).
- Supports three base motion integration models:
 - Move forward then rotate.
 - Rotate then move forward.
 - Rotate and move simultaneously.
- Updates full kinematic chain including the mobile base using extended DH parameters.

4.4 Exercise1.py

- Initializes the mobile manipulator and sets up a task list including joint limit and end-effector position tasks.
- Implements recursive Task-Priority control with null-space projection.
- Uses DLS to compute control velocities.
- Visualizes robot motion, end-effector trajectory, and task error using Matplotlib.

4.5 Exercise2.py

- Replaces position tasks with a single configuration task.
- Applies the Weighted Damped Least-Squares (WDLS) method.
- Changes the end-effector goal randomly in every run.
- Logs and visualizes:
 - Norm of position and orientation errors.
 - Velocity output for all degrees of freedom.

4.6 Exercise3.py

- Evaluates the accuracy of different base motion models.
- Uses predefined target positions for consistency across simulations.
- Compares three motion models:
 - Forward then rotate.
 - Rotate then forward.
 - Simultaneous rotate and move.
- Stops the simulation when all targets are reached.
- Logs and saves both end-effector and base positions for comparison.

4.7 plot.py

- Loads saved trajectory data (.npy files) for different base integration models.
- Plots the X-Y paths of the end-effector and base using different colors and line styles.
- Annotates the plot with legends, axis labels, and grid for visual clarity.

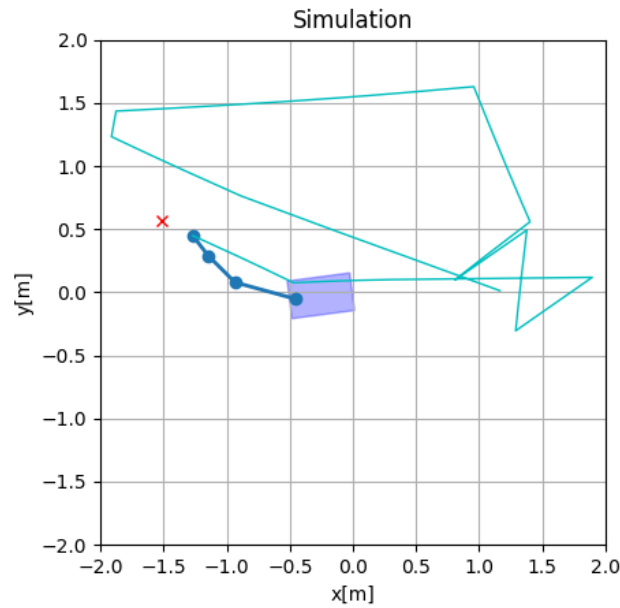
5 Results and Discussion

The result of this lab are obtained as:

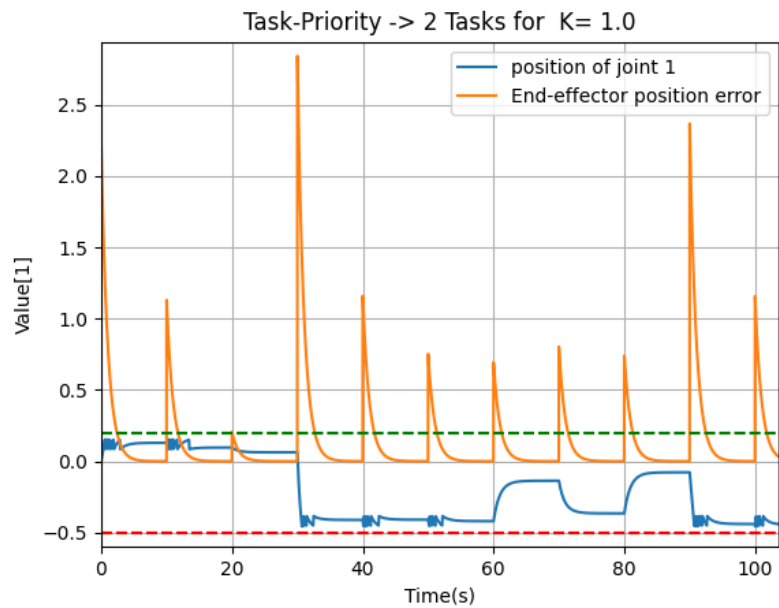
5.1 Exercise 1: Joint Limit Task and Position Control

In this exercise, the recursive Task-Priority control framework was tested using a combination of two tasks:

- A **joint limit task**, implemented as a set-based inequality constraint to prevent the third joint from exceeding a predefined velocity range.
- An **end-effector position task**, responsible for guiding the manipulator's tip to randomly selected 2D targets.



Simulation of the mobile manipulator, including end-effector goal.



Evolution of the TP control error and joint position over time.

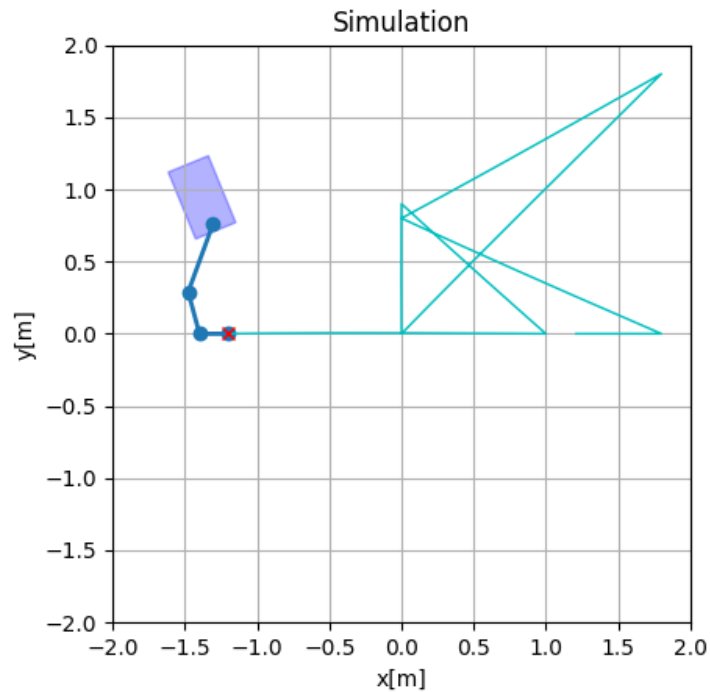
Figure 2: Exercise 1: Joint limit handling with end-effector positioning.

Discussion:

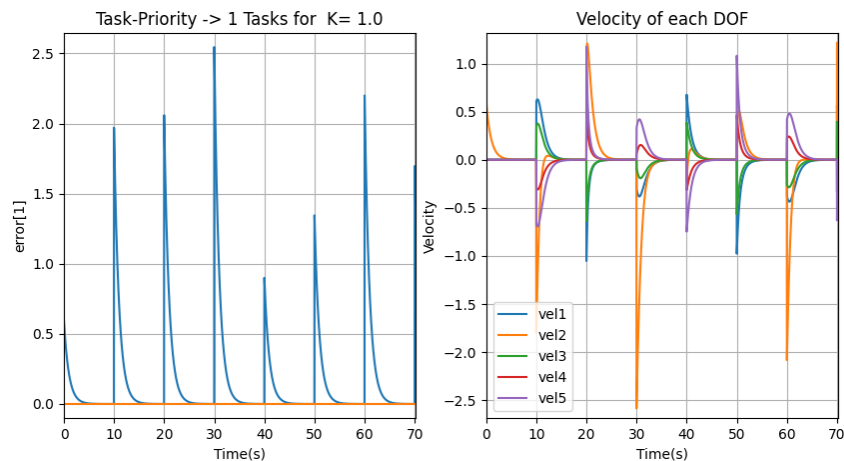
- The system demonstrated stable convergence to randomly generated goals while strictly enforcing the joint velocity limits.
- The task error plot shows repeated peaks in the position error, corresponding to new goal assignments. These are followed by smooth error reduction as the manipulator approaches the targets.

5.2 Exercise 2: Configuration Control with Weighted DLS

This exercise focused on controlling the full configuration of the end-effector, comprising both its position and orientation in 2D space. A Weighted Damped Least-Squares (WDLS) approach was employed, enabling the system to prioritize certain joints over others when computing control commands.



Simulation of the mobile manipulator, executing configuration control.



Left: Task error evolution. Right: Joint velocities over time.

Figure 3: Exercise 2: Weighted DLS-based configuration control.

Discussion:

- The error plot shows a consistent decrease in task error after each new goal is introduced, confirming convergence of the controller.
- The velocity plot reveals how different joints respond with varying magnitudes, highlighting the impact of the applied weight matrix.

- some joints show suppressed or amplified motion, demonstrating that WDLS successfully enforced the desired prioritization among degrees of freedom.

5.3 Exercise 3: Comparison of Base Integration Strategies

This exercise aimed to compare the behavior of the mobile manipulator under three different mobile base integration strategies:

1. Move forward then rotate
2. Rotate then move forward
3. Rotate and move forward simultaneously (arc-based)

A common set of five predefined target positions for the end-effector was used across all trials to ensure consistency in comparison.

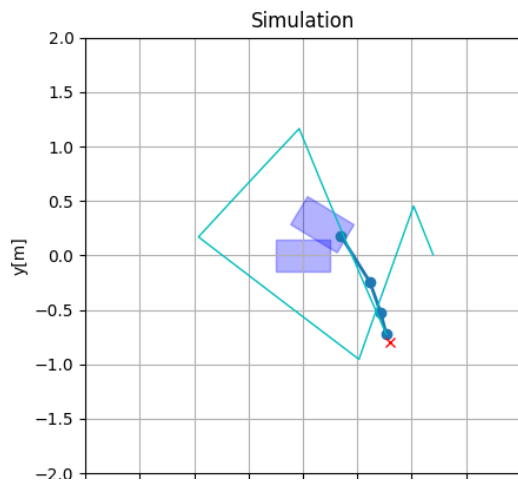
Figure 4 shows the system behavior when using the **move forward then rotate** strategy. The manipulator successfully reached the goals, and the control error remained stable.

Target End-Effector Positions

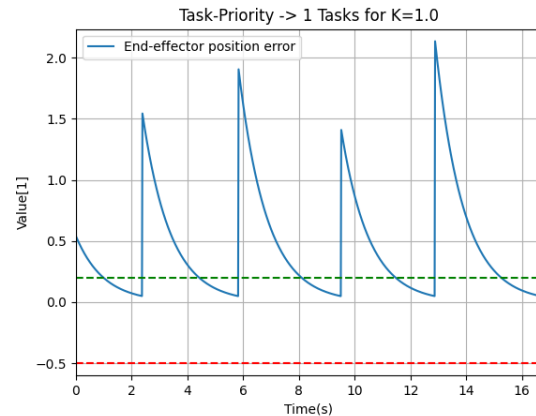
The following 2D goal positions were used as end-effector targets for all experiments in Exercise 3:

Target Index	x [m]	y [m]
1	1.0	0.5
2	0.5	-1.0
3	-1.0	0.2
4	0.0	1.2
5	0.8	-0.8

Table 1: Predefined end-effector target positions for Exercise 3



Simulation of the mobile manipulator using the forward-then-rotate base strategy.



Task-Priority control error for end-effector position over time.

Figure 4: Exercise 3a: Behavior using “Move Forward Then Rotate” strategy.

Figure 5 summarizes the trajectory comparison across all three motion integration methods. Both base and end-effector paths are plotted to visually contrast how each strategy affects motion behavior.

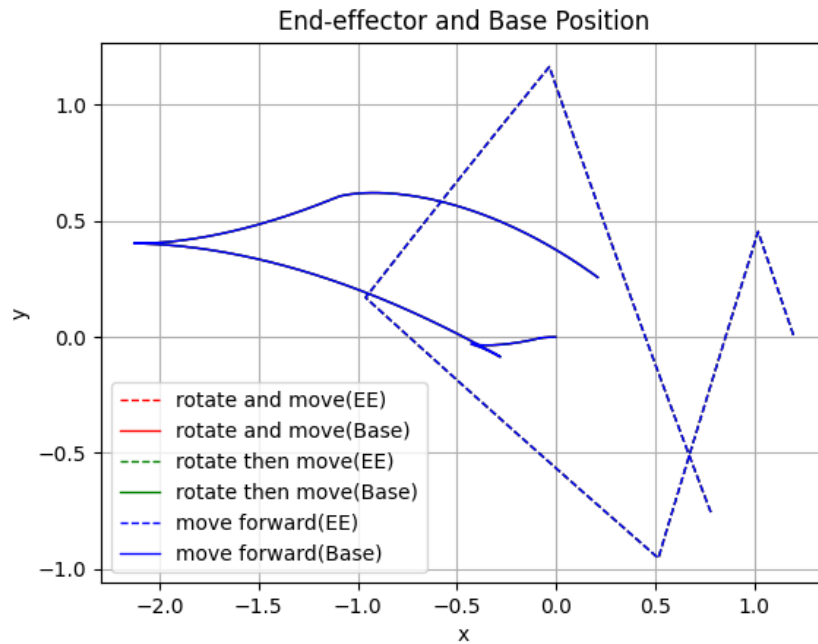


Figure 5: Overlay of end-effector and base trajectories for all integration strategies.

6 Conclusion

This lab successfully extended the Task-Priority (TP) kinematic control framework to a mobile manipulator that integrates a differential-drive base with a 3-link planar arm. **In Exercise 1**, the TP controller was used to reach random end-effector targets while respecting joint velocity limits. **In Exercise 2**, we introduced the Weighted Damped Least-Squares (WDLS) method to influence which joints contributed more to the motion. The weighting strategy was effective—certain joints moved more or less depending on their assigned priority. **In Exercise 3** evaluated how different methods for integrating the mobile base’s motion affected the overall trajectory.

Hence, the lab provided a practice in controlling mobile manipulators using recursive TP control, task hierarchy, and mobility-aware kinematics.

Appendix

This appendix contains the source code used in the implementation of Lab 6 exercises.

A lab6_robotics.py

```

1  from lab5_robotics import *
2  import math
3  deg90 = np.pi / 2
4  class MobileManipulator:
5      """
6      Constructor.
7
8      Arguments:
9      d (Numpy array): list of displacements along Z-axis
10     theta (Numpy array): list of rotations around Z-axis
11     a (Numpy array): list of displacements along X-axis
12     alpha (Numpy array): list of rotations around X-axis
13     revolute (list of Bool): list of flags specifying if the
14         corresponding joint is a revolute joint
15     """
16     def __init__(self, d, theta, a, alpha, revolute):
17         self.d = d
18         self.theta = theta
19         self.a = a
20         self.alpha = alpha
21         self.revolute = revolute
22
23         # List of joint types extended with base joints
24         self.revoluteExt = [True, False] + self.revolute
25
26         self.r = 0.2 # Distance from robot centre to manipulator
27                     # base
28         self.dof = len(self.revoluteExt) # Number of DOF of the
29                     # system
30
31         # Vector of joint positions (manipulator)
32         self.q = np.zeros((len(self.revolute), 1))
33
34         # Vector of base pose (position & orientation)
35         self.eta = np.zeros((3, 1))
36
37         # Initialise robot state
38         self.update(np.zeros((self.dof, 1)), 0.0)
39
40     """
41     Method that updates the state of the robot.
42
43     Arguments:
44     dQ (Numpy array): a column vector of quasi velocities
45     dt (double): sampling time

```

```

44     """
45
46     def update(self, dQ, dt):
47         # Update manipulator
48         self.q += dQ[2:, 0].reshape(-1, 1) * dt
49         for i in range(len(self.revolute)):
50             if self.revolute[i]:
51                 self.theta[i] = self.q[i]
52             else:
53                 self.d[i] = self.q[i]
54
55         # Update mobile base pose
56         forward_vel = dQ[1, 0]
57         angular_vel = dQ[0, 0]
58         yaw = self.eta[2, 0]
59
60         # Use one at a time:
61         #self.forward_rotate(dQ, dt)           # 1. Move forward then
62         rotate
63         self.rotate_forward(dQ, dt)           # 2. Rotate then move
64         forward
65         #self.rotateandmove_forward(dQ, dt) # 3. Rotate and move
66         forward (arc)
67
68     # Base kinematics
69     x, y, yaw = self.eta.flatten()
70     Tb = self.translation(x, y) @ self.rotation_z(yaw) #
71     Transformation of the mobile base
72     # Tb = self.rotation_z(yaw) @ self.translation(x, y)
73
74     ### Additional rotations performed, to align the axis:
75     # Rotate Z +90 (using the theta of the first base joint)
76     # Rotate X +90 (using the alpha of the first base joint)
77     ## Z now aligns with the forward velocity of the base
78     # Rotate X -90 (using the alpha of the second base joint)
79     ## Z is now back to vertical position
80     # Rotate Z -90 (using the theta of the first manipulator
81     joint)
82
83     # Modify the theta of the base joint, to account for an
84     additional Z rotation
85     self.theta[0] -= deg90
86
87     # Combined system kinematics (DH parameters extended with
88     base DOF)
89     dExt = np.concatenate([np.array([0, self.r]), self.d])
90     thetaExt = np.concatenate([np.array([deg90, 0]), self.theta.
91     reshape(3,)]])
92     aExt = np.concatenate([np.array([0, 0 ]), self.a])
93     alphaExt = np.concatenate([np.array([deg90, -deg90]), self.
94     alpha])

```

```
88         self.T = kinematics(dExt, thetaExt, aExt, alphaExt, Tb)
89
90         """
91         Method that returns the characteristic points of the robot.
92         """
93
94     def drawing(self):
95         return robotPoints2D(self.T)
96
97         """
98         Method that returns the end-effector Jacobian.
99         """
100
101     def getEEJacobian(self):
102         return jacobian(self.T, self.revoluteExt)
103
104         """
105         Method that returns the end-effector transformation.
106         """
107
108     def getEETransform(self):
109         return self.T[-1]
110
111         """
112         Method that returns the position of a selected joint.
113
114         Argument:
115         joint (integer): index of the joint
116
117         Returns:
118         (double): position of the joint
119         """
120
121     def getJointPos(self, joint):
122         return self.q[joint - 2]
123
124     def getBasePose(self):
125         return self.eta
126
127         """
128         Method that returns number of DOF of the manipulator.
129         """
130
131     def getDOF(self):
132         return self.dof
133
134     ###
135     def getLinkJacobian(self, link):
136         return jacobianLink(self.T, self.revoluteExt, link)
137
138     def getLinkTransform(self, link):
139         return self.T[link]
140     def getOrientationJacobian(self, link):
```

```

141         return self.getLinkJacobian(link)[5, :] # Orientation
           Jacobian
142     '''
143     Method that returns the configuration Jacobian.
144     '''
145     def getConfigurationJacobian(self , link):
146
147         J = np.block([[self.getLinkJacobian(link)[0:2, :].reshape
           (2,5)],
148                       [self.getOrientationJacobian(link).reshape(1,5)
           ]])
149
150         return J
151     def get2dbaseTransformation(self):
152         x, y, yaw = self.eta.flatten()
153         return np.array([[np.cos(yaw), -np.sin(yaw), x],
154                          [np.sin(yaw), np.cos(yaw), y],
155                          [0, 0, 1]])
156     def get2dbaseTransformation(self):
157         x, y, yaw = self.eta.flatten()
158         return np.array([[np.cos(yaw), -np.sin(yaw), x],
159                          [np.sin(yaw), np.cos(yaw), y],
160                          [0, 0, 1]])
161     def translation(self, x, y):
162         return np.array([[1, 0, 0 , x],
163                          [0, 1, 0 , y],
164                          [0, 0, 1 , 0],
165                          [0, 0, 0 , 1]])
166     def rotation_z(self, theta):
167         return np.array([[np.cos(theta), -np.sin(theta), 0, 0],
168                          [np.sin(theta), np.cos(theta), 0, 0],
169                          [0, 0, 1, 0],
170                          [0, 0, 0, 1]])
171
172     def rotate_forward(self,dQ , dt):
173         #Update mobile base pose ( rotate then move forward)
174         self.eta[2,0] += dQ[0, 0]* dt
175         self.eta[0,0] += dQ[1, 0]*np.cos(self.eta[2,0])*dt
176         self.eta[1,0] += dQ[1, 0]*np.sin(self.eta[2,0])*dt
177
178     def rotateandmove_forward(self,dQ , dt):
179         # Update mobile base pose ( rotate and move forward)
180         if math.isclose(dQ[0,0], 0.0):
181             pass
182         else:
183             #calculate radius
184             self.radius = dQ[1,0]/dQ[0,0]
185             self.eta[0,0] += -self.radius*np.sin(self.eta[2,0]) +
186                             self.radius*np.sin(dQ[0,0]*dt+(self.eta[2,0]))
187             self.eta[1,0] += self.radius*np.cos(self.eta[2,0]) -
188                             self.radius*np.cos(dQ[0,0]*dt+(self.eta[2,0]))
189             self.eta[2,0] += dQ[0,0]*dt
190
191     def forward_rotate(self,dQ,dt):

```

```

189         # Update mobile base pose (move forward then rotate)
190         self.eta[0,0] += dQ[1, 0]*np.cos(self.eta[2,0])*dt
191         self.eta[1,0] += dQ[1, 0]*np.sin(self.eta[2,0])*dt
192         self.eta[2,0] += dQ[0, 0]* dt

```

B Exercise1.py

```

1  from lab6_robotics import * # Includes numpy import
2  import matplotlib.pyplot as plt
3  import matplotlib.patches as patch
4  import matplotlib.animation as anim
5  import matplotlib.transforms as trans
6
7  # Robot model
8  d      = np.zeros(3)          # displacement along Z-axis
9  theta  = np.array([0, 0., 0.]).reshape(3,1) # rotation around Z-axis
10 alpha  = np.zeros(3)          # rotation around X-axis
11 a      = np.array([0.5, 0.3 , 0.2]) # displacement along X-axis
12 revolute = [True, True , True]   # flags specifying the type of
    joints
13 robot = MobileManipulator(d, theta, a, alpha, revolute)
14
15 # Desired end-effector position
16 sigma_endeff = np.zeros((2,1))
17 # Desired end-effector configuration
18 sigma_config = np.block([[sigma_endeff], [0]]).reshape(3,1)
19
20 velocity_limit = [-0.5,0.2] # Joint velocity limits
21 # Task definition
22 obstacle_tasks = []
23 JointLimit_link = 3
24 tasks = [
25     JointLimit("Joint_limit", velocity_limit, velo_thershold =
26               [0.05 , 0.1] , link = JointLimit_link ),
27     Position2D("End-effector_position", np.array([1.0, 0.5]).
28               reshape(2,1), 5)
29 ]
30 num_obstacles = len(obstacle_tasks)
31 err_counter = 0 # Error counter
32 error_vector = np.zeros((0 ,len(tasks))) # Error vector
33 obstacle_dist = np.zeros((0,0))
34 # Simulation params
35 dt = 1.0/60.0
36 # Drawing preparation
37 fig = plt.figure()
38 ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim
39                     =(-2,2))
40 ax.set_title('Simulation')
41 ax.set_aspect('equal')
42 ax.grid()
43 ax.set_xlabel('x[m]')
44 ax.set_ylabel('y[m]')

```

```

42 rectangle = patch.Rectangle((-0.25, -0.15), 0.5, 0.3, color='blue',
    alpha=0.3)
43 veh = ax.add_patch(rectangle)
44 line, = ax.plot([], [], 'o-', lw=2) # Robot structure
45 path, = ax.plot([], [], 'c-', lw=1) # End-effector path
46 point, = ax.plot([], [], 'rx') # Target
47 PPx = []
48 PPy = []
49
50 # Simulation initialization
51 def init():
52     global tasks , sigma_endeff , error_vector
53     line.set_data([], [])
54     path.set_data([], [])
55     point.set_data([], [])
56     sigma_endeff = np.random.uniform(-2, 2, (2, 1))
57     # Desired end-effector configuration
58     sigma_config = np.block([[sigma_endeff], [np.pi/2]]).reshape(3,1)
59     # set desired values for each task in each iteration
60     # endeffector task and configuration task
61     for task in tasks:
62         if(task.name == "End-effector_position"):
63             task.setDesired(sigma_endeff)
64         elif(task.name == "End-effector_configuration"):
65             task.setDesired(sigma_config)
66     return line, path, point
67
68 # Simulation loop
69 def simulate(t):
70     global tasks
71     global robot , err_counter , obstacle_dist
72     global PPx, PPy , sigma_endeff , error_vector
73
74     ### Recursive Task-Priority algorithm
75
76     dof = robot.getDOF() # Get robot degrees of freedom
77     P = np.eye(dof) ## Initialize null-space projector
78     err_acc = np.zeros((1,0))
79     obs_dis = np.zeros((1, 0))
80     dq = np.zeros((dof,1)) # Initialize joint velocity vector
81
82     for index, task in enumerate(tasks): # Loop over tasks
83         task.update(robot) # Update task state
84         J = task.getJacobian() # Compute augmented Jacobian
85         # Compute task velocity
86         if(task.isTaskActive()):# check activation of the task
87             Jbar= J@P # Compute augmented Jacobian
88             dq = dq + DLS(Jbar,0.1)@( task.getFeedforwardVelocity() +
89                 task.getGain()@task.getError() - J@dq)
90             P = P - np.linalg.pinv(Jbar)@Jbar # Update null-space
91                 projector
92             new_error = np.linalg.norm(task.getError()) #error
93                 accumulation

```

```

92         if task.name == "Joint_limit":
93             new_error = robot.getJointPos(JointLimit_link-1) # get
94                 joint position and store
95         elif task.name == "Obstacle_avoidance":
96             # get distance to obstacle and store
97             obs_dis = np.block([ obs_dis , task.distance_to_obstacle
98                 ()])
99
100             err_acc = np.block([ err_acc , new_error]) # Accumulate
101                 velocity
102
103         # Update robot
104         robot.update(dq, dt)
105         error_vector = np.block([[error_vector],[ err_acc]])
106         obstacle_dist = np.block([[obstacle_dist], [obs_dis]])
107         err_counter += 1
108
109         # Update drawing
110         # -- Manipulator links (skip base links)
111         PP = robot.drawing()[ :, 2:] # Skip the 3 base links
112         line.set_data(PP[0,:], PP[1,:])
113         PPx.append(PP[0,-1])
114         PPy.append(PP[1,-1])
115         path.set_data(PPx, PPy)
116         point.set_data(sigma_endeff[0], sigma_endeff[1])
117         # -- Mobile base
118         eta = robot.getBasePose()
119         veh.set_transform(trans.Affine2D().rotate(eta[2,0]) + trans.
120             Affine2D().translate(eta[0,0], eta[1,0]) + ax.transData)
121
122         return line, veh, path, point
123
124 # Run simulation
125 animation = anim.FuncAnimation(fig, simulate, np.arange(0, 10, dt),
126     interval=10, blit=True, init_func=
127     init, repeat=True)
128 plt.show()
129
130 # Drawing The Error of each task with respect time
131 total_time = np.arange(0,err_counter*dt,dt)
132 print(err_counter , error_vector.shape , obstacle_dist.shape)
133 ylim =( min(error_vector.min() , error_vector.min() , velocity_limit
134     [0] ) - 0.1 ,max(error_vector.max(),
135     error_vector.max(), velocity_limit [1] ) + 0.1)
136 fig = plt.figure()
137 ax2 = fig.add_subplot(111, autoscale_on=False , xlim = (0,err_counter
138     *dt),
139     ylim=ylim)
140 ax2.set_title('Task-Priority->' + str(len(tasks)) + 'Tasks' + "for
141     K=")

```

```

137         + str(tasks[0].getGain()[0][0]))
138 ax2.set_xlabel('Time(s)')
139 ax2.set_ylabel('Value[1]')
140 ax2.grid()
141
142 # Drawing the error of each task with respect to time
143 for i in range(num_obstacles):
144     ax2.plot(total_time, obstacle_dist[ : ,i ], label="d_" + str(
145         i)
146             + " 'distance to obstacle' ")
147
148 for i,task in enumerate(tasks):
149     if(task.name == "Joint_limit"):
150         # Add dashed lines for min and max velocity
151         ax2.plot(total_time, error_vector[ : ,i ], label="position_of
152             joint_1" )
153         ax2.axhline( y=velocity_limit[0], color='r', linestyle='
154             dashed', )
155         ax2.axhline( y=velocity_limit[1], color='g', linestyle='
156             dashed', )
157     elif(task.name not in ["Obstacle_avoidance"]):
158         ax2.plot(total_time, error_vector[ : ,i ], label=task.name + "
159             error" )
160
161 # maximum and minimum velocity limits drawing dash
162 ax2.axhline( y=velocity_limit[0], color='r', linestyle='dashed', )
163 ax2.axhline( y=velocity_limit[1], color='g', linestyle='dashed', )
164 ax2.legend()
165 plt.show()

```

C Exercise2.py

```

1 from lab6_robotics import * # Includes numpy import
2 import matplotlib.pyplot as plt
3 import matplotlib.patches as patch
4 import matplotlib.animation as anim
5 import matplotlib.transforms as trans
6
7 # Robot model
8 d = np.zeros(3) # displacement along Z-axis
9 theta = np.array([0, 0., 0.]).reshape(3,1) # rotation around Z-axis
10 alpha = np.zeros(3) # rotation around X-axis
11 a = np.array([0.5, 0.3 , 0.2]) # displacement along X-axis
12 revolute = [True, True , True] # flags specifying the type of
13     joints
14
15 robot = MobileManipulator(d, theta, a, alpha, revolute)
16
17 W = np.diag([1,1,1,1,1])*0.01# Weight matrix
18 velocity_vector = np.zeros((0,5))
19 # Desired end-effector position
20 sigma_endeff = np.zeros((2,1))
21 # Desired end-effector configuration

```

```

20 sigma_config = np.block([[sigma_endeff], [0]]).reshape(3,1)
21
22 velocity_limit = [0,0] # Joint velocity limits
23 # Task definition
24 obstacle_tasks = []
25 JointLimit_link = 3
26 tasks = [
27     # JointLimit("Joint limit", velocity_limit,
28                 velo_threshold = [0.05 , 0.1] , link =
29                 JointLimit_link ),
30     # Position2D("End-effector position", np.array([1.0,
31             0.5]).reshape(2,1), 5),
32     Configuration2D("End-effector configuration", sigma_config
33                     .reshape(3,1) , link = 5),
34 ]
35 num_obstacles = len(obstacle_tasks)
36 err_counter = 0 # Error counter
37 error_vector = np.zeros((0 ,len(tasks))) # Error vector
38 ore_error =[]
39 obstacle_dist = np.zeros((0,0))
40 # Simulation params
41 dt = 1.0/60.0
42
43 # Drawing preparation
44 fig = plt.figure()
45 ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim
46                     =(-2,2))
47 ax.set_title('Simulation')
48 ax.set_aspect('equal')
49 ax.grid()
50 ax.set_xlabel('x[m]')
51 ax.set_ylabel('y[m]')
52 rectangle = patch.Rectangle((-0.25, -0.15), 0.5, 0.3, color='blue',
53                             alpha=0.3)
54 veh = ax.add_patch(rectangle)
55 line, = ax.plot([], [], 'o-', lw=2) # Robot structure
56 path, = ax.plot([], [], 'c-', lw=1) # End-effector path
57 point, = ax.plot([], [], 'rx') # Target
58 PPx = []
59 PPy = []
60
61 # Simulation initialization
62 index = 0
63 def init():
64     global tasks , sigma_endeff , error_vector , index
65     line.set_data([], [])
66     path.set_data([], [])
67     point.set_data([], [])
68
69     sigma_endeff = np.random.uniform(-2, 2, (2, 1))
70     # Desired end-effector configuration
71     sigma_config = np.block([[sigma_endeff], [np.pi/2]]).reshape(3,1)
72     # set desired values for each task in each iteration

```

```

67     # endeffector task and configuration task
68     sigma_endeff_array = np.array
        ([[1,1.2],[-1,-1.2],[1.8,0],[0,0.8],[1.8,1.8],[0,0],[0,0.9],[1,0],[-1.2,0]])

69     sigma_endeff = sigma_endeff_array[index].reshape(2,1)
70     sigma_config = np.block([[sigma_endeff], [0]]).reshape(3,1)
71     for task in tasks:
72         if(task.name == "End-effector_position"):
73             task.setDesired(sigma_endeff)
74         elif(task.name == "End-effector_configuration"):
75             task.setDesired(sigma_config)
76     index += 1
77     if index == len(sigma_endeff_array):
78         index = 0
79     return line, path, point
80
81
82 # Simulation loop
83 def simulate(t):
84     global tasks
85     global robot , err_counter , obstacle_dist
86     global PPx, PPy , sigma_endeff , error_vector , ore_error ,
        velocity_vector
87
88     ### Recursive Task-Priority algorithm
89
90     dof = robot.getDof() # Get robot degrees of freedom
91     P = np.eye(dof) ## Initialize null-space projector
92     err_acc = np.zeros((1,0))
93     obs_dis = np.zeros((1 , 0))
94     dq = np.zeros((dof,1)) # Initialize joint velocity vector
95
96
97     for index, task in enumerate(tasks): # Loop over tasks
98         task.update(robot) # Update task state
99         J = task.getJacobian() # Compute augmented Jacobian
100         # Compute task velocity
101         if(task.isTaskActive()):# check activation of the task
102             Jbar= J@P # Compute augmented Jacobian
103             dq = dq + W_DLS(Jbar,0.1,W)@( task.getFeedforwardVelocity
                () +
                    task.getGain()@task.getError() - J@dq)
104             P = P - np.linalg.pinv(Jbar)@Jbar # Update null-space
                projector
105             new_error = np.linalg.norm(task.getError()) #error
                accumulation
106
107
108             if task.name == "Joint_limit":
109                 new_error = robot.getJointPos(JointLimit_link-1) # get
                    joint position and store
110                 err_acc = np.block([ err_acc , new_error]) # Accumulate
                    velocity
111             elif task.name == "Obstacle_avoidance":

```

```

112         # get distance to obstacle and store
113         obs_dis = np.block([ obs_dis , task.distance_to_obstacle
                               ()])
114         elif task.name == "End-effector_configuration":
115             pose_error = np.linalg.norm(task.getError()[0:2,:]) #
                error accumulation
116             ore_error.append(np.linalg.norm(task.getError()[2,0])) #
                Accumulate orientation error
117             err_acc = np.block([ err_acc , pose_error]) #
                Accumulate velocity
118         else:
119             err_acc = np.block([ err_acc , new_error]) # Accumulate
                velocity
120
121     # Update robot
122     robot.update(dq, dt)
123     error_vector = np.block([[error_vector],[ err_acc]])
124     obstacle_dist = np.block([[obstacle_dist], [obs_dis]])
125     err_counter += 1
126
127     velocity_vector = np.block([[velocity_vector],[dq.T]])
128
129     # Update drawing
130     # -- Manipulator links
131     PP = robot.drawing()[ :, 2:] # Skip the 3 base links
132     line.set_data(PP[0,:], PP[1,:])
133     PPx.append(PP[0,-1])
134     PPy.append(PP[1,-1])
135     path.set_data(PPx, PPy)
136     point.set_data(sigma_endeff[0], sigma_endeff[1])
137     # -- Mobile base
138     eta = robot.getBasePose()
139     veh.set_transform(trans.Affine2D().rotate(eta[2,0]) + trans.
        Affine2D().translate(eta[0,0], eta[1,0]) + ax.transData)
140
141     return line, veh, path, point
142
143 # Run simulation
144 animation = anim.FuncAnimation(fig, simulate, np.arange(0, 10, dt),
145                               interval=10, blit=True, init_func=
        init, repeat=True)
146 plt.show()
147
148 # Drawing The Error of each task with respect time
149 total_time = np.arange(0,err_counter*dt,dt)
150
151 ylim =( min(error_vector.min() , error_vector.min() , velocity_limit
        [0] ) - 0.1 ,max(error_vector.max(),
        error_vector.max(), velocity_limit [1] ) + 0.1)
152 fig , ax = plt.subplots(1,2 , figsize=(10,5))
153
154 # ax[0] = fig.add_subplot(111, autoscale_on=False , xlim = (0,
        err_counter*dt),

```

```

156 #                                     ylim=ylim)
157 ax[0].set_autoscale_on(False)
158 ax[0].set_xlim(0,err_counter*dt)
159 ax[0].set_ylim(ylim)
160 ax[0].set_title('Task-Priority->' + str(len(tasks)) + 'Tasks' + "
    for K="
161                 + str(tasks[0].getGain()[0][0]))
162 ax[0].set_xlabel('Time(s)')
163 ax[0].set_ylabel('error[1]')
164 ax[0].grid()
165
166 # Drawing the error of each task with respect to time
167 for i in range(num_obstacles):
168     ax[0].plot(total_time, obstacle_dist[ : ,i ], label="d_" +
        str(i)
169                + "'distance to obstacle'")
170
171 for i,task in enumerate(tasks):
172
173     if(task.name == "End-effector_configuration"):
174         ax[0].plot(total_time, error_vector[ : ,i ], label="position_
            of_end-effector" )
175         ax[0].plot(total_time, ore_error, label="orientation_of_end-
            effector" )
176
177     elif(task.name == "Joint_limit"):
178         # Add dashed lines for min and max velocity
179         ax[0].plot(total_time, error_vector[ : ,i ], label="position_
            of_joint1" )
180         ax[0].axhline( y=velocity_limit[0], color='r', linestyle='
            dashed', )
181         ax[0].axhline( y=velocity_limit[1], color='g', linestyle='
            dashed', )
182
183     elif(task.name not in ["Obstacle_avoidance"]):
184         ax[0].plot(total_time, error_vector[ : ,i ], label=task.name
            + "_error" )
185
186 # maximum and minimum velocity limits drawing dash
187 # ax[0].axhline( y=velocity_limit[0], color='r', linestyle='dashed',
188 #               )
189 # ax[0].axhline( y=velocity_limit[1], color='g', linestyle='dashed',
190 #               )
191 # ax[0].legend()
192
193 # Drawing the velocity of each DOF with respect to time
194 ylim =( min(velocity_vector.min(), velocity_vector.min() ) - 0.1 ,
        max(velocity_vector.max(),
            velocity_vector.max(), ) + 0.1)
195 ax[1].set_autoscale_on(False)
196 ax[1].set_xlim(0,err_counter*dt)
197 ax[1].set_ylim(ylim)
198 ax[1].set_title('Velocity_of_each_DOF')

```

```

198 ax[1].set_xlabel('Time(s)')
199 ax[1].set_ylabel('Velocity')
200 ax[1].grid()
201
202 for i in range(5):
203     ax[1].plot(total_time, velocity_vector[ : ,i ], label="vel" + str
                (i+1) )
204
205 ax[1].legend()
206
207 plt.show()

```

D Exercise3.py

```

1  from lab6_robotics import * # Includes numpy import
2  import matplotlib.pyplot as plt
3  import matplotlib.patches as patch
4  import matplotlib.animation as anim
5  import matplotlib.transforms as trans
6
7  # Robot model
8  d      = np.zeros(3)          # displacement along Z-axis
9  theta  = np.array([0, 0., 0.]).reshape(3,1) # rotation around Z-axis
10 alpha  = np.zeros(3)          # rotation around X-axis
11 a      = np.array([0.5, 0.3 , 0.2]) # displacement along X-axis
12 revolute = [True, True , True]    # flags specifying the type of
    joints
13 robot = MobileManipulator(d, theta, a, alpha, revolute)
14
15 W = np.diag([1000,1000,1000,1000,1000])*0.001 # Weight matrix
16
17 # Predefined end-effector targets for consistent testing
18 sigma_targets = [
19     np.array([[1.0], [0.5]]),
20     np.array([[0.5], [-1.0]]),
21     np.array([[-1.0], [0.2]]),
22     np.array([[0.0], [1.2]]),
23     np.array([[0.8], [-0.8]])
24 ]
25 current_target_idx = 0
26 max_targets = len(sigma_targets)
27 target_reached_threshold = 0.05 # Acceptable error norm to consider
    target reached
28
29 # Initialize target
30 sigma_endeff = sigma_targets[current_target_idx]
31 sigma_config = np.block([[sigma_endeff], [0]]).reshape(3, 1)
32
33 velocity_limit = [-0.5, 0.2]
34 JointLimit_link = 3
35 obstacle_tasks = []
36

```

```

37
38 # Task list
39 tasks = [
40     Position2D("End-effector_position", sigma_endeff, 5),
41     # Configuration2D("End-effector configuration", sigma_config,
42         link=5),
43 ]
44
45 # Logging and simulation state
46 num_obstacles = len(obstacle_tasks)
47 err_counter = 0
48 error_vector = np.zeros((0, len(tasks)))
49 ore_error = []
50 obstacle_dist = np.zeros((0, 0))
51 endeff_pose = np.zeros((2, 0))
52 base_pose = np.zeros((2, 0))
53 dt = 1.0 / 60.0
54
55 # Drawing preparation
56 fig = plt.figure()
57 ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim
58     =(-2,2))
59 ax.set_title('Simulation')
60 ax.set_aspect('equal')
61 ax.grid()
62 ax.set_xlabel('x[m]')
63 ax.set_ylabel('y[m]')
64 rectangle = patch.Rectangle((-0.25, -0.15), 0.5, 0.3, color='blue',
65     alpha=0.3)
66 veh = ax.add_patch(rectangle)
67 line, = ax.plot([], [], 'o-', lw=2)
68 path, = ax.plot([], [], 'c-', lw=1)
69 point, = ax.plot([], [], 'rx')
70 PPx = []
71 PPy = []
72
73 # Initialization
74 def init():
75     global tasks, sigma_endeff, error_vector
76     line.set_data([], [])
77     path.set_data([], [])
78     point.set_data([], [])
79     sigma_endeff = sigma_targets[current_target_idx]
80     sigma_config = np.block([[sigma_endeff], [0]]).reshape(3,1)
81
82     for task in tasks:
83         if task.name == "End-effector_position":
84             task.setDesired(sigma_endeff)
85         elif task.name == "End-effector_configuration":
86             task.setDesired(sigma_config)
87
88     return line, path, point

```

```

87
88 # Simulation loop
89 def simulate(t):
90     global current_target_idx, sigma_endeff, sigma_config
91     global robot, err_counter, error_vector, ore_error, obstacle_dist
92     , endeff_pose, base_pose
93     global PPx, PPy
94
95     if current_target_idx >= max_targets:
96         return line, veh, path, point # Stop simulation
97
98     dof = robot.getDOF()
99     P = np.eye(dof)
100     dq = np.zeros((dof, 1))
101     err_acc = np.zeros((1, 0))
102     obs_dis = np.zeros((1, 0))
103
104     for task in tasks:
105         task.update(robot)
106         J = task.getJacobian()
107
108         if task.isTaskActive():
109             Jbar = J @ P
110             dq += W_DLS(Jbar, 0.1, W) @ (
111                 task.getFeedforwardVelocity() + task.getGain() @ task
112                 .getError() - J @ dq
113             )
114             P -= np.linalg.pinv(Jbar) @ Jbar
115
116             err = np.linalg.norm(task.getError())
117             err_acc = np.block([err_acc, err])
118
119     error_vector = np.block([[error_vector], [err_acc]])
120     obstacle_dist = np.block([[obstacle_dist], [obs_dis]])
121     err_counter += 1
122     # Check if target reached
123     if np.linalg.norm(tasks[0].getError()) < target_reached_threshold
124         :
125         current_target_idx += 1
126         if current_target_idx < max_targets:
127             sigma_endeff = sigma_targets[current_target_idx]
128             sigma_config = np.block([[sigma_endeff], [0]]).reshape(3,
129                 1)
130             tasks[0].setDesired(sigma_endeff)
131             return line, veh, path, point
132
133     robot.update(dq, dt)
134
135     endeff_pose = np.hstack((endeff_pose, robot.getEETransform()
136         [0:2,3].reshape(2,1)))
137     base_pose = np.hstack((base_pose, robot.getBasePose()[0:2,0].
138         reshape(2,1)))

```

```

134
135
136     # Update drawing
137     # -- Manipulator links
138     PP = robot.drawing()[ :, 2:] # Skip the 3 base links
139     line.set_data(PP[0,:], PP[1,:])
140     PPx.append(PP[0,-1])
141     PPy.append(PP[1,-1])
142     path.set_data(PPx, PPy)
143     point.set_data(sigma_endeff[0], sigma_endeff[1])
144     # -- Mobile base
145     eta = robot.getBasePose()
146     veh.set_transform(
147         trans.Affine2D().rotate(eta[2,0]) +
148         trans.Affine2D().translate(eta[0,0], eta[1,0]) +
149         ax.transData
150     )
151
152     return line, veh, path, point
153
154 # Run animation
155 animation = anim.FuncAnimation(fig, simulate, np.arange(0, 10, dt),
156                               interval=10, blit=True, init_func=init
157                               , repeat=True)
158
159 plt.show()
160
161 # Plot error over time
162 total_time = np.arange(0, err_counter * dt, dt)
163 print(err_counter, error_vector.shape, obstacle_dist.shape)
164 ylim = (
165     min(error_vector.min(), velocity_limit[0]) - 0.1,
166     max(error_vector.max(), velocity_limit[1]) + 0.1
167 )
168 fig = plt.figure()
169 ax2 = fig.add_subplot(111, autoscale_on=False, xlim=(0, err_counter *
170 dt), ylim=ylim)
171 ax2.set_title('Task-Priority->' + str(len(tasks)) + 'Tasks_for_K='
172 + str(tasks[0].getGain()[0][0]))
173 ax2.set_xlabel('Time(s)')
174 ax2.set_ylabel('Value[1]')
175 ax2.grid()
176
177 for i in range(num_obstacles):
178     ax2.plot(total_time, obstacle_dist[:, i], label="d_" + str(i) + "
179         'distance_to_obstacle'")
180
181 for i, task in enumerate(tasks):
182     if task.name == "End-effector_configuration":
183         ax2.plot(total_time, error_vector[:, i], label="position_of_
184             end-effector")
185         ax2.plot(total_time, ore_error, label="orientation_of_end-
186             effector")
187     elif task.name == "Joint_limit":

```

```

181     ax2.plot(total_time, error_vector[:, i], label="position_of_
        joint_1")
182     ax2.axhline(y=velocity_limit[0], color='r', linestyle='dashed
        ')
183     ax2.axhline(y=velocity_limit[1], color='g', linestyle='dashed
        ')
184     elif task.name not in ["Obstacle_avoidance"]:
185         ax2.plot(total_time, error_vector[:, i], label=task.name + "
            error")
186
187 ax2.axhline(y=velocity_limit[0], color='r', linestyle='dashed')
188 ax2.axhline(y=velocity_limit[1], color='g', linestyle='dashed')
189 ax2.legend()
190 plt.show()
191
192 # Save simulation data
193 print("End-effector_shape:", endeff_pose.shape)
194 print("Base_pose_shape:", base_pose.shape)
195
196 # 1. move forward then rotate
197 #np.save('move forward then rotate.npy', np.hstack((endeff_pose.T,
        base_pose.T)))
198
199 # 2. rotate then move forward
200 #np.save('rotate then move forward.npy', np.hstack((endeff_pose.T,
        base_pose.T)))
201
202 # 3. rotate and move forward
203 np.save('rotate_and_move_forward.npy', np.hstack((endeff_pose.T,
        base_pose.T)))

```

E plot.py

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import os
4
5  # Always use the path where this script is located
6  base_path = os.path.dirname(os.path.abspath(__file__))
7
8  # Load from same folder as this script
9  a_load = np.load(os.path.join(base_path, 'rotate_and_move_forward.npy
        '))
10 b_load = np.load(os.path.join(base_path, 'rotate_then_move_forward.
        npy'))
11 c_load = np.load(os.path.join(base_path, 'move_forward_then_rotate.
        npy'))
12
13 print("'rotate_and_move_forward.npy:", a_load.shape)
14 print("rotate_then_move_forward.npy:", b_load.shape)
15 print("move_forward_then_rotate.npy:", c_load.shape)
16

```

```
17 fig = plt.figure()
18 ax = fig.add_subplot(111)
19
20 ax.plot(a_load[:,0], a_load[:,1], label='rotate_and_move(EE)', color
        ='r', linestyle='dashed', lw=1)
21 ax.plot(a_load[:,2], a_load[:,3], label='rotate_and_move(Base)',
        color='r', lw=1)
22
23 ax.plot(b_load[:,0], b_load[:,1], label='rotate_then_move(EE)',
        color='g', linestyle='dashed', lw=1)
24 ax.plot(b_load[:,2], b_load[:,3], label='rotate_then_move(Base)',
        color='g', lw=1)
25
26 #ax.plot(c_load[:,0], c_load[:,1], label='move forward(EE)', color
        ='b', linestyle='dashed', lw=1)
27 #ax.plot(c_load[:,2], c_load[:,3], label='move forward(Base)', color='b
        ', lw=1 )
28
29 ax.set_xlabel('x')
30 ax.set_ylabel('y')
31 ax.set_title('End-effector_and_Base_Position')
32 ax.grid()
33 ax.legend()
34
35 plt.show()
```